



Shop IBM



Support



Downloads

[IBM Home](#)[Products](#)[Consulting](#)[Industries](#)[News](#)[About IBM](#)[IBM](#) : [developerWorks](#) : [Components](#) : [Components articles](#)

developerWorks

The OO design process: Use cases applied, Part 1



Specifying a use case

[Allen Holub](#) (allen@holub.com)

Chief Technical Officer, NetReliance

April 2001

In this month's article I continue from last month's article on use-case planning by starting to fill out the use-case template for our first (Depositing funds) use case. I've not just filled in the template, but also provided extensive comments about my thought processes as I was working.

A few preliminary comments about how the use-case document is built are in order. I've ordered the sections of the template by considering how the template will be used. You'll rarely create the template in the same order that you use it, though. I've presented the use case in the template order, but I was skipping around in the form when I actually put the thing together. I started out with the name and description, then skipped down and worked on the scenarios a bit, then skipped down and added an implementation note, then skipped back up and worked on the scenarios some more. Then, I filled in the Desired outcome section, and skipped down and added a postcondition, and around and around I went.

Moreover, in a real-world situation, it's rare that a person would develop an entire use case singlehandedly. Ideally, the business-related components (the use-case name, the description, the scenarios, the user goals, perhaps the desired outcome) would be developed primarily by the marketing side, with engineering in a consulting role. The technical items (dependencies, preconditions and postconditions, formal workflow analysis, implementation requirements and notes) would come primarily from engineering, with the marketing organization in a consulting role. Business rules and Requirements are typically created jointly. It's even better if a real-live customer -- an actual user of the product -- participates in the process, too.

You'll also work on several design artifacts in parallel. It's natural to uncover hitherto unknown aspects of the problem as you work on the use cases, and you should go back and add these to the problem statement before you forget them. Keeping the various documents that comprise the design in phase with each other is a real problem with a large design, but it is critical. As soon as you discover some flaw in your thinking about a problem, you have to change *all* the relevant documents immediately. I typically have the problem statement, any relevant use-case descriptions, the UML diagrams, and the code all in front of me at the same time as I work. It's also important to be able to roll back to previous designs if you go down a wrong path, though I don't know of any automated document-management packages that make rollback truly painless since many types of document -- drawings, formatted text, ASCII source code, etc. -- comprise the design, and they might all be affected by the change. (If anybody knows of a document-management system that's up to this task, [send me e-mail](#).)

Now let's look at the first use case.

An example use case

Search [Advanced](#) [Help](#)

Contents:

[An example use case](#)[Resources](#)[About the author](#)[Rate this article](#)

Related dW content:

[Use case planning](#)[Use cases, an Introduction](#)

Name	1.0. Kid depositing funds into account
Description	A kid (customer) deposits money into his or her account. Deposits must be authorized by a bank-officer/parent before they go into effect.

This is the first time that the notion of authorization has popped up in this context, and it's an example of how writing a use case can find flaws in the problem statement. During my first pass on this use case, I wrote "parents aren't involved in this use case," but my next thought was "but what prevents the kid from just depositing arbitrary sums of money?" The notion of authorization solves the problem, but introduces a few interesting scenarios, which I'll discuss below.

The original problem statement did cover security issues, but not in a way that's workable. Here's the relevant portion from the original:

Original problem statement

There are a few security issues. Like a real bank, kids can't make a deposit or withdrawal by modifying their passbooks; they have to ask the bank to do it. The bank processes the transaction and updates the passbook. That is, the kid gives money to the bank, which puts it into the account. The bank takes money out of the account, and dispenses it to the kid. (Practically speaking, this means that only parents can make deposits or withdrawals, but parents should process withdrawal requests when asked or the kids will assume that the bank is a ruse for not letting them get at their own money.)

In retrospect, that business of the bank taking money out of an account and dispensing it doesn't work, because banks don't do that (tellers do, bank officers do, banks do not). So, I now go back and modify the problem statement to bring the notion of parent approval of certain transactions more in line with what I've discovered in the use case.

Here's the revised section:

Revised problem statement

There are a few security issues. Only a bank officer (that is, a parent) can change the interest rate, set up a new account (and the initial balance), or close an account.

Though, for the most part, kids can use the bank entirely on their own, some transactions (deposits and loans) require parent/bank officer approval before they can be completed. Transactions like this will be visible to kids, and will show up in the passbook, but unapproved deposits are not available for withdrawal.

Withdrawals don't require parent approval. A withdrawal request will cause a check to be printed. These checks work like real-world checks. If it's lost, then a stop-payment order can be issued (for a fee). The check can be presented to the bank (a parent) for conversion to cash.

The issue of checks came to me as I was writing -- yet another example of how the design process works. You discover missing pieces of the problem as you look at the problem from different perspectives.

Of course, having made a change to the problem statement, I now need to get domain-expert approval before I can check in the new version. (Since I'm my own domain expert in the current case, that's easy.)

Desired outcome	The account balance gets larger

User goals	Kid: To make the account balance as large as possible and to watch wealth accumulate Parent: To make sure that kids don't inflate the account balance with an arbitrary deposits to which they're not entitled
------------	--

Note that different users of the system have different goals, and that these goals are often mutually exclusive. Kids want the account balance to be arbitrarily large, but parents want to assure that only real funds -- funds actually available to the kids -- are deposited. This sort of goal clash is commonplace, and one of the points of the current exercise is to identify (and address) these clashes.

Participants/Roles

I'll identify the roles using the CRC-card format I discussed last month. This list, again, is developed over time as I work on the scenarios; what you see here is the final form of the list.

Note that some of the participants are automated (the Teller) and others are real people. Also note that some of the participants are passive objects in the system (Passbook). This is one place where a use case that's intended for program development might vary from one that's used for UI development. A UI-style use case would typically be interested only in the roles taken on by the physical users of the system. In OO systems, though, even objects that you think of as passive have responsibilities and operations defined on them. (For example, the passbook might print itself.)

Class	Responsibilities	Collaborators
Kid (bank customer)	● Fills out deposit slip. ● Holds the passbook.	Teller (handles the Kid's requests)
Parent (bank officer)	Authorizes deposits	Teller (requests deposits from)
Teller	● Solicits deposit slip. ● Gets permission to make deposit from bank officer. ● Updates passbook.	Parent (authorizes transactions for), Kid (makes requests of)
Account	????	????
Passbook	Tracks transaction history	Kid (looks at it), Teller (updates it)
Deposit slip	Identifies the amount of the deposit	Kid (fills it out), Parent (authorizes it), Teller (transfers it to Passbook), Passbook (records it)

Remember, the organization I've presented here is more for the convenience of the user of the use case than the creator of the use case. Surprisingly, I can't think of anything for objects of the Account class to do in this system. The current balance is held by the passbook, which also keeps a record of the transactions on the account. I suspect, then, that the account and the passbook are the same thing; all that the Account does is assign some identifying information to the transaction record. But if the kid/customer is holding their passbook, then there's no need to store the identifying information in the account as well; you can just ask kids for their passbooks to find out what accounts they own.

This issue really highlights the difference between a database-style, entity-relationship (ER) worldview and an object-oriented (OO) worldview. Mapping one worldview to the other is called *object-relational mapping*, and that mapping is rarely straightforward, since you need to reconcile two conflicting organizational strategies. It's rare that a good object model will make a good database schema (or vice versa). In the current example, an

ER worldview might consider Customers, Accounts, and Passbooks all to be valid entities. The Customer contains customer-specific identification information and identifies various accounts that the customer owns, the Passbook contains transaction histories, and the Account contains account-specific information like current balance. The object model doesn't work that way for reasons I just discussed. Since there's nothing for an Account to *do*, it's not a legitimate class.

In an OO system, a class shouldn't exist unless there's something for it to *do*; there must be responsibilities (operations). In fact, a class is defined solely by what it can do -- the data that it uses to perform those tasks is an irrelevant implementation detail that should be hidden from the rest of the world. An account might encapsulate data, but I can't think of any operations that it will perform, thus Account is not a proper class. (See [Resources](#) for more on this way of looking at OO systems.)

Put another way, object modelers and entity modelers have fundamentally different worldviews. Object modelers are interested almost exclusively in operations -- what the objects do. The state data that an object must hold to implement those operations is usually irrelevant this early in the design process. A static attribute is of interest to an object modeler only if it can be used to distinguish one class of objects from another. (For example, employees have salaries, people don't, so the salary attribute serves as a distinguishing characteristic of Employee.) Database entity modelers, however, concern themselves solely with the static attributes -- the operations are immaterial to them. The net result is that organizations that are anathema to entity modelers (classes with no state data at all, one-to-one and many-to-many relationships between classes, etc.) appear all over the place in OO systems.

The final culture-clash issue surrounds the organization of the static data. In an OO system, for example, the members of a committee would effectively be contained within the Committee class. That is, the members would be static attributes of the Committee class. In a relational system, it's more likely that a Committee identifier would be an attribute of the member (so that you could join or select on a committee name).

But I digress. The foregoing notwithstanding, I'll keep "Account" in the list for the time being in case some operation on an account surfaces. I suspect that Account will eventually go away, though.

So that's about half of the template -- in a way, the least significant half since these parts of the template are more useful in organizing my thoughts than in actually specifying what's going on in the use case. Next time I'll finish up by providing the more significant parts of the use-case definition: the scenarios and the activity diagrams. These parts actually attempt to map out the work flow as the use cases progress.

Resources

- This article continues my series on the OO-design process. The first six parts are:
 - [Getting started](#)
 - [Beginning to design software](#)
 - [Refining the problem definition](#)
 - [Verifying the analysis](#)
 - [Use cases, an Introduction](#)
 - [Use case planning](#)
- Workflow diagrams (along with the rest of UML) are described nicely in Martin Fowler and Kendall Scott's [UML Distilled](#), 2nd Ed.

About the author

Allen Holub is the CTO at NetReliance, a San-Francisco-based company that's building a secure global infrastructure for conducting trusted business transactions over the net. (Yes, we're hiring.) He has worked in the computer industry since 1979, and is widely published. Allen has [eight books](#) to his credit, the latest of which covers the traps and pitfalls of Java threading ([Taming Java Threads](#) [Berkeley: Apress, 2000; ISBN 1893115100]).

He's been designing and building OO software for longer than he cares to remember (in C++ and Java). He teaches OO-Design and Java for the University of California, Berkeley, Extension (since 1982). Contact Allen at allen@holub.com or at allen.holub@net-reliance.com.

Allen is also a composer, artist, and instrument-rated commercial pilot. He holds B.A. degrees from the University of California, Berkeley, in Medieval History and Computer Science.



What do you think of this article?

Killer! (5) Good stuff (4) So-so; not bad (3) Needs work (2) Lame! (1)

Comments?